

不确定有穷状态自动机的确定化

目录

一、实验名称	2
二、实验目的	2
三、实验原理	2
1、NFA 定义	2
2、DFA 的定义	2
3、closure 函数	2
4、move 函数	3
四、实验思路	3
1、输入	3
2、closure 算法	3
3、move 算法	3
4、构造子集	4
5、输出	4
五、实验小结	4
1、输入存储问题	4
2、closure 算法问题	4
3、输出问题	5
六、附件	5
1、源代码	5
2、运行结果截图	7

一、实验名称

不确定有穷状态自动机的确定化

二、实验目的

输入：非确定有穷状态自动机 NFA

输出：确定化的有穷状态自动机 DFA

三、实验原理

1、NFA 定义

一个不确定的有穷自动机 M 是一个五元组， $M=(K,E,f,S,Z)$ 其中

K 是一个有穷集，它的每个元素称为一个状态；

E 是一个有穷字母表，它的每个元素称为一个输入符号；

f 是一个从 $K \times E^*$ 到 K 的子集的映像，即： $K \times E^* \rightarrow 2^K$ ，其中 2^K 表示 K 的幂集；

S 包含于 K ，是一个非空初态集；

Z 包含于 K ，是一个终态集。

2、DFA 的定义

一个确定的有穷自动机 M 是一个五元组， $M=(K,E,f,S,Z)$ 其中

K 是一个有穷集，它的每个元素称为一个状态；

E 是一个有穷字母表，它的每个元素称为一个输入符号；

f 是转换函数，是 $K \times E \rightarrow K$ 上的映像，即，如 $f(k_i, a) = k_j (k_i \in K, k_j \in K)$ 就意味着，当前状态为 k_i ，输入字符为 a 时，将转换到下一状态 k_j ，我们把 k_j 称作 k_i 的一个后继状态；

$S \in K$ ，是唯一的一个初态；

Z 包含于 K ，是一个终态集，终态也称可接受状态或结束状态。

3、closure 函数

状态集合 I 的 ϵ -闭包，表示为 $\epsilon\text{-closure}(I)$ ，定义为一状态集，是状态集 I 中的任何状态 S 经任意条 ϵ 弧而能到达的状态的集合。

4、move 函数

状态集合 I 的 a 弧转换，表示为 $\text{move}(I,a)$ ，定义为状态集合 J ，其中 J 是所有那些从 I 中的某一状态经过一条 a 弧而到达的状态的全体。

四、实验思路

本次实验采用 python 完成。

1、输入

根据课本 NFA 的定义，输入五元组，依次输入状态集、输入符号、初态集、终态集以及映像，将这些分别存入五个列表中。其中关于映像的输入格式：先输入状态一，再输入输入符号，最后输入状态二，一次输入一条弧。

2、closure 算法

定义 closure 函数形式为 $\text{closure}(a,f)$ ，其中， a 为要做 closure 闭包的状态集合， f 为 NFA 的映像的集合。具体思想为：

- a. 设立一个最终返回结果的列表 b ，初值与列表 a 相等。设立一个空列表 s ，用于存放每次 closure 闭包新加入的状态。
- b. 执行 while 循环，此循环判断条件为 1，即会一直执行下去，直到遇到 closure 闭包没有新增状态的时候执行结束。
- c. 对 a 中的每一个状态求 closure 闭包，即判断 f 中状态一等于 a 中状态的弧，再判断 f 中该状态的弧是否为 ϵ （具体代码中用 '\$' 代替），若是，则将该弧的状态二加入 s 中。
- d. 判断 s 是否为空，若为空则说明此次循环没有新增状态，即说明 closure 闭包在上一次循环时已执行完毕，输出上次循环的结果 b 。若 s 不为空，说明本次循环仍然有新增状态，则将新增状态加入 b 中，并且将新增的状态集合赋值给 a ，以新增的状态集继续做循环判断，直到某次循环 s 为空结束。

3、move 算法

move 算法的核心思想与 closure 算法一致，其函数形式为 $\text{move}(a,e,f)$ ，其中 e 为 move 算法 $\text{move}(l,a)$ 的 a 。move 算法只要求从状态集合中某一状态经过一条 a 弧而到达的状态全体，所以不需要进行 while 循环执行多次，只需执行 closure 算法中 c 步骤一次即可。

4、构造子集

建立两个列表 $C1$ 、 $C2$ ，其中 $C1$ 用于存放最终的状态集， $C2$ 作为标记使用，对应 $C1$ 中的子集，若 $C1$ 中的子集也进行了 closure 闭包则 $C2$ 中相应元素标记为 1，否则为 0。具体思想为：

- a. 首先对初态集进行 closure 闭包，存于 $C1$ 中， $C2$ 的第一个元素赋值为 0。
- b. 标记 $C2$ 第一个元素为 1，对 $C1$ 中第一个集合先做 move 算法再做 closure 算法，若其中一个算法得出空集合则直接返回空列表，否则判断 $C1$ 中是否有该状态集，若无则加入 $C1$ 中， $C2$ 中相应元素赋值为 0，表示未标记。
- c. 重复执行 b 步骤，直到 $C2$ 中所有元素为 1，表示标记完毕，执行完成，得到 $C1$ 为最终状态子集。

5、输出

采用矩阵形式输出， $C1$ 中每个状态集合的下标为最终合并后的状态。

五、实验小结

本次实验注意以下问题：

1、输入存储问题

若根据课本形式应输入 $M=(K,E,f,S,Z)$ ，再对 f 进行展开，虽然用算法实现这一形式不难，但是对于后续的操作不太方便，所以最终选择了依次输出五元组，分别存于五个列表中。

2、closure 算法问题

最初想用递归的思想实现 closure 算法，即每次进行一步 closure 闭包，返回结果为新得到的状态集的 closure 闭包，但是对于递归结束的判断条件以及参数的传递不太明确，所以最终没有选择递归，而是选择了死循环里面加上退出循

环条件的形式完成。

3、输出问题

输出的形式最终没有实现 DFA 的状态图而是使用矩阵的形式输出，问题在于对于以状态集合为结点构造状态图这样的图形形式方面的知识不了解，最终以矩阵形式输出。

通过本次实验，对于 NFA 转换为 DFA 的过程有了深刻的认识，对于 closure 算法和 move 算法的思想非常清楚。

六、附件

1、源代码

```
K = [] # 状态
E = [] # 符号
f = [] # 弧
S = [] # 初态
Z = [] # 终态

# 输入
print('E21414020 陈国柱')
a = input('输入状态(以空格区分,以换行结束):')
K = a.split(' ')
a = input('输入输入符号(以空格区分,以换行结束):')
E = a.split(' ')
a = input('输入初态(以空格区分,以换行结束):')
S = a.split(' ')
a = input('输入终态(以空格区分,以换行结束):')
Z = a.split(' ')
print('输入弧的条数:')
n = int(input())
print('输入弧(分别输入状态 1,输入符号,状态 2,以空格区分换行结束,e表示为$)')
for i in range(n):
    f.append([])
    a = input()
    f[len(f)-1] = a.split(' ')

# closure 算法
def closure(a, f): # a 为列表
    b = a
```

```

while 1:
    s = []
    for i in a:
        for j in range(len(f)):
            if i == f[j][0] and f[j][1] == '$':
                s.append(f[j][2])

    if len(s) == 0:
        break
    else:
        for i in s:
            b.append(i)
        a = s
    return sorted(b)

# move 算法
def move(a, e, f): # a 为列表 e 为一个符号
    s = []
    for i in a:
        for j in range(len(f)):
            if i == f[j][0] and f[j][1] == e:
                s.append(f[j][2])
    return sorted(s)

# 算出最终子集
C1 = [] # C1 为最终子集
C2 = []
C1.append(closure(S, f))
C2.append(0)
while C2.pop(len(C2)-1) == 0:
    C2.append(0)
    for i in range(len(C1)):
        if C2[i] == 0:
            C2[i] = 1
            for j in E:
                A = move(C1[i], j, f)
                if A == []:
                    break
                B = closure(A, f)
                if B == []:
                    break
            k = 0
            for m in C1:
                if B == m:
                    k = k+1

```

```

        if k == 0:
            C1.append(B)
            C2.append(0)
print('输出 NFA 构造的子集:')
print(C1)
print('输出 DFA:')
print('S', end='    ')
for x in E:
    print(x, end='    ')
print('\n')
# 输出 DFA
for i in range(len(C1)):
    print(i, end='    ')
    for j in E:
        a1 = move(C1[i], j, f)
        if a1 == []:
            print(a1, end='    ')
            continue
        a2 = closure(a1, f)
        if a2 == []:
            print(a2, end='    ')
            continue
        for k in range(len(C1)):
            if C1[k] == a2:
                print(k, end='    ')
                break
    y = 0 # 判断子集中是否有终态
    for m in Z:
        if m in C1[i]:
            y = y+1
    if y == 0:
        print(0)
    else:
        print(1)
print('\n')

```

2、运行结果截图